

Installing and Docker (Important) :
Please read and follow this so we can proceed with our practical :
https://drive.google.com/file/d/12W-D1aHGiMjcNV3Dn7mYByVMlUgiBtJ5/view?usp=drive_link

Alt : https://docs.docker.com/engine/install/debian/

—*(Not related to practical)*---
Docker CLI Cheat Sheet (Extra for those who want to learn more) :
https://drive.google.com/file/d/11tebsR4vEYcxA-Wn1wBuoi_KI3TXeukM/view?usp=drive_link

Installing Docker (Fixed) :
Add Docker's official GPG key:

```
#1)
sudo apt-get update

#2)
sudo apt-get install ca-certificates curl

#3)
sudo install -m 0755 -d /etc/apt/keyrings

#4)
curl -fsSL https://download.docker.com/linux/debian/gpg | sudo gpg --dearmor
-o /etc/apt/keyrings/docker.gpg

#5)
sudo chmod a+r /etc/apt/keyrings/docker.gpg

#6)
CODENAME=$( . /etc/os-release && echo $VERSION_CODENAME )
ARCH=$(dpkg --print-architecture)

#7)
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null <<EOF
deb [arch=${ARCH} signed-by=/etc/apt/keyrings/docker.gpg]
https://download.docker.com/linux/debian ${CODENAME} stable
EOF

#8)
sudo sed -i 's/kali-rolling/bookworm/' /etc/apt/sources.list.d/docker.list

#9)
sudo apt-get update

#10)
sudo apt-get install docker-ce docker-ce-cli containerd.io
docker-buildx-plugin docker-compose-plugin

#Test wether docker are functioning properly.
#1)
sudo systemctl status docker
#2)
docker version
#3)
docker compose version
```

If an error pops out after installation, precisely like this (at the bottom of the installation script) :
Errors were encountered while processing:
/tmp/apt-dpkg-install-IzyoRC/4-docker-buildx-plugin_0.29.1-1~debian.12~bookworm_amd64.deb E: Sub-process /usr/bin/dpkg returned an error code (1)

Fix :

```
# 1. Remove the old conflicting Kali package
sudo apt remove -y docker-buildx

# 2. Clean up partial installs
sudo apt --fix-broken install -y

# 3. Retry the Docker install
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
```

Practical : Hosting a simple Web Challenge using Docker

1) Create our working folder/directory :

```
#Make a directory
mkdir Docker101/Web && cd Docker101/Web
```

2) Create a Dockerfile (use “nano Dockerfile”) :

```
#Create a file named "Dockerfile" and paste these :
FROM python:3.11-slim
RUN useradd -m -u 1000 ctfuser
WORKDIR /home/ctf/app
COPY app.py .
RUN pip install --no-cache-dir flask
RUN chown -R ctfuser:ctfuser /home/ctf
USER ctfuser
EXPOSE 8000
CMD ["python","app.py"]
```

3) Create app.py (use “nano app.py”) :

```
#Create another file named "app.py" and paste this :
from flask import Flask
app = Flask(__name__)
@app.route("/")
def home():
   return "FLAG{web_minimal}"
if __name__=="__main__":
   app.run(host="0.0.0.0", port=8000)
```

4) Build and run it using Docker :
   - For Linux

```
#Run these in sequence :
#1)
docker build -t ctf-web .
#2)
docker run -d --name ctf-web -p 15010:8000 ctf-web

# test
firefox http://127.0.0.1:15010/
```

   - For Windows

```
#Run these in sequence :
#1)
docker build -t ctf-web .
#2)
docker run -d --name ctf-web -p 15010:8000 ctf-web

#Open in Chrome :
http://127.0.0.1:15010/
```

```
#Clean Up ( After finish practicals )
#1)
docker rm -f ctf-web
#2)
docker image rm ctf-web
```

Practical 2 : Hosting a “listener” challenges or script only challenges

1) Create our working folder/directory :

```
#Create a folder (cd Docker101) - if you haven't yet.

mkdir TCP-Chal && cd TCP-Chal
```

2) Create a Dockerfile (use “nano Dockerfile”):

```
#Create a file named "Dockerfile"
FROM node:20-alpine
RUN addgroup -S ctf && adduser -S -G ctf ctf
WORKDIR /home/ctf
COPY server.js .
RUN chown -R ctf:ctf /home/ctf
USER ctf
EXPOSE 1337
CMD ["node","server.js"]
```

3) Create a server.js (use “nano server.js”) :

```
#paste into server.js
const net = require('net');
const srv = net.createServer(s=> {
 s.write("Welcome. Send 'flag'\\n");
 s.on('data', d => {
   if (d.toString().trim()==='flag') s.write('FLAG{tcp_minimal}\\n');
   else s.write('nope\\n');
 });
});
srv.listen(1337,'0.0.0.0');
```

4) Build and run it using Docker :

- For Linux

```
docker build -t ctf-tcp .
docker run -d --name ctf-tcp -p 15133:1337 ctf-tcp
# test
echo flag | nc 127.0.0.1 15133
```

- For Windows

```
docker build -t ctf-tcp .
docker run -d --name ctf-tcp -p 15133:1337 ctf-tcp
# test (with ncat)
echo flag | ncat 127.0.0.1 15133
```

```
# cleanup
docker rm -f ctf-tcp
docker image rm ctf-tcp
```

Setting up our vulnerable websites (for web vulnerabilities practise), for this we are using a ready-made vulnerable web made by GMI's Alumni, the goated Sir Firdaus Khairuddin : https://github.com/firdauskhairuddin/lekir-docker

```
#Clone Lekir's repo
git clone https://github.com/firdauskhairuddin/lekir-docker.git
cd lekir-docker

docker compose build

#To start the website use :
docker compose up

#To stop the website :
docker compose down

Access it via :
http://localhost:1337 / http://127.0.0.1:1337
```